



Bioinformatics Programming Project

Discussion Guide & Workflow



Discussion Schedule

Day: Thursday | Location: Office 408

Group	Time
A1	10:00 AM
A2	11:00 AM
A3	12:00 PM
A4	1:00 PM

1 Data Acquisition & Project Setup

Establish the working environment and prepare the dataset (either by downloading from SRA or via Simulation). Data must represent **more than one class** (e.g., Healthy vs. Disease).

Task 1: Project Structure & Environment Setup

- Create a structured project directory (`data/`, `src/`, `results/`).
- Create a virtual environment (`venv` or `conda`).
- Install required packages (`biopython`, `numpy`, `pandas`) and generate `requirements.txt`.

Deliverable: `requirements.txt` & `README.md`

Task 2: Data Sourcing (FASTQ Files)

- Option A:** Download real FASTQ files related to multiple classes using NCBI SRA Toolkit (`prefetch` & `fasterq-dump`).
- Option B:** Simulate FASTQ data for multiple classes.

Deliverable: Raw `.fastq` files in `data/` directory

2 Custom Python Parsing & QC

Implement custom scripts to understand FASTQ file structure and compute basic QC metrics using the provided code.

Task 3: FASTQ Parsing

- Implement a parser using Biopython to read FASTQ files and extract read ID, sequence, and quality scores.

Deliverable: `src/fastq_parser.py`

Task 4: Custom Quality Control Computation

- Compute basic QC metrics: Total reads, average read length, GC content, Q20/Q30 percentages, and per-base quality.

Deliverable: `src/quality_control.py` & generated `.txt` report

3 Dockerized QC & Trimming Pipeline

Run standard bioinformatics QC tools using Docker to clean the data and generate trusted reports.

Task 5: Dockerized FastQC & Fastp Pipeline

- Run **FastQC** on raw data.
- Run **Fastp** to trim adapters, filter low-quality reads, and generate `fastp.json` and `fastp.html` reports.
- Run FastQC again on the trimmed data.
- Run **MultiQC** to aggregate all reports.

Deliverable: Trimmed `.fastq` files, `fastp.json`, `multiqc_report.html`

4 Feature Extraction & K-mers (Core Requirement) 🎯

This phase is crucial. Students must prove their understanding of the features extracted from reports and store them systematically.

Task 6: Feature Extraction from Fastp Report

- Parse the `fastp.json` file generated in Phase 3.
- Extract key statistical features (e.g., before/after filtering reads, Q20/Q30 rates, GC content, duplication rates).
- Important:** Students must understand the biological/technical meaning of every feature they extract.

Deliverable: `src/feature_extraction.py`

Task 7: K-mer Frequency Extraction

- Write a Python script to extract **K-mer frequencies** (e.g., 3-mers or 4-mers) from the FASTQ sequences.
- Definition to know:* K-mers are short, overlapping subsequences of length 'k'. They are crucial features for ML as they capture sequence composition and can differentiate between genomic regions or organism classes.

Deliverable: `features.csv` (Rows = Samples, Cols = Features + K-mers + Class Label)

5 Machine Learning (Optional/Bonus)

For students who finish early and want an extra challenge.

Task 8: Machine Learning Classification

- Load the `features.csv`.
- Train a basic classifier (e.g., Random Forest or Logistic Regression) to predict the "Class" of the samples based on extracted features and K-mers.
- Evaluate the model (Accuracy, Precision, Recall).

Deliverable: `src/ml_model.py` & performance summary

Optional Task



Evaluation Criteria (What you will be asked)



Data Understanding: Why did you choose this SRA dataset? What are the classes present in it?



FASTQ Format: What is the difference between Sequence and Quality score? What does the Phred score mean?



Code Comprehension: Explain the functions in `fastq_parser.py` and `quality_control.py`. What does each function return and why did we use Biopython?



Feature Knowledge (Crucial): What does Q30 mean? What is GC content and why might it be abnormal? What are K-mers and why are they used as features for ML?



Docker: What is the purpose of Volume mapping (`-v`) in Docker? Why did we run Fastp after FastQC?